
LECTURE NOTES

Lecture 15: Discrete Optimization

University

Simon Holm
April - 2026



DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

1	Randomized Optimization Heuristics	3
1.1	Examples to illustrate concepts	3
2	Search Methods	5
2.1	Constraint Handling	6
2.2	A Classification	6
2.3	Approaches to ROHs	6
2.4	Components of a Search Algorithm	7
3	Constructive Search	8
3.1	Complete Tree Search	8
3.2	Exploiting Information	8
3.3	Greedy algorithms	8
3.4	Incomplete Search algorithms	8
3.5	Bounded backtrack	9
3.6	Incomplete Search Ideas	9
3.7	Construction-Based Metaheuristics	9

1 Randomized Optimization Heuristics

Until now we have mostly seen complete methods, which explore all possibilities. incomplete methods use heuristics to approach “good enough” solution.

These include

- effective rules without theoretical support (such as genetics)
- trial and error

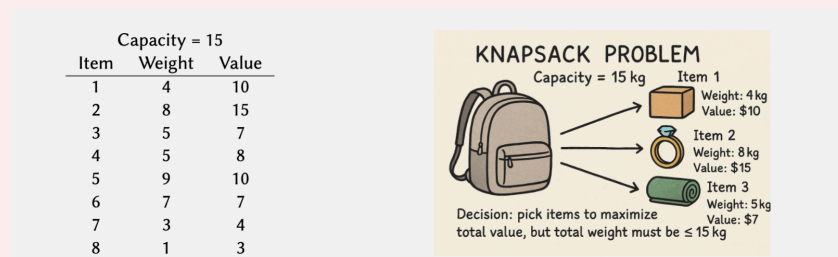
Overall the use of heuristics is a basis on empirical evidence rather than mathematical logic. Getting things done in the given time.

1.1 Examples to illustrate concepts

Example The Knapsack Problem

Given: A ground set of items with weights and values. A knapsack with a weight capacity.

Task: Find the subset that maximizes the total value



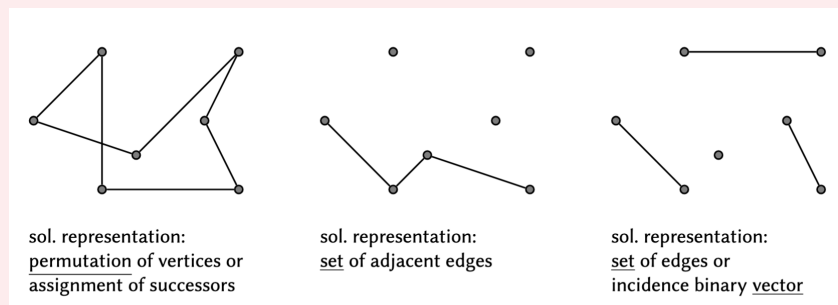
Solution Set: $\{1, 2, 7\}$ or Incidence vector: $[1, 1, 0, 0, 0, 0, 1, 0]$

Total weight: 15; Total value: 26

Example The Traveling Salesman Problem

Given: A graph $G = (V, E)$ and a weight function $\omega : E \rightarrow \mathbb{R}$.

Task: Find the shortest Hamiltonian tour.



This extends a partial solution until either a complete solution is found or the solution is good enough.

Example The Single Machine Total Weighted Tardiness

Given: a set of n jobs $\{1, \dots, n\}$ to be processed on a single machine and for each job j a processing time p_j , a weight w_j and a due date d_j

Task: Find a schedule that minimizes the total weighted tardiness $\sum_{j=1}^n w_j \cdot T_j$, where $T_j = \max\{C_j - d_j, 0\}$ (C_j completion time of job j)

Job	1	2	3	4	5	6
Processing Time	3	2	2	3	4	3
Due date	6	13	4	9	7	17
Weight	2	3	1	5	1	2

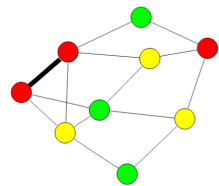
Sol. representation:
Linear permutation
 $\phi = [3, 1, 5, 4, 1, 6]$

Job	3	1	5	4	2	6
C_j	2	5	9	12	14	17
T_j	0	0	2	3	1	0
$w_j \cdot T_j$	0	0	2	15	3	0

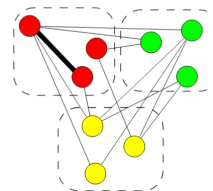
Example The Graph Vertex-Coloring

Given: A graph G and a set of colors Γ . A **proper coloring**: each vertex receives a color and no two adjacent vertices receive the same color.

Task: Find a proper coloring of G that uses the minimal number of colors (chromatic number).



Assignment representation



Partitioning representation

2 Search Methods

Problem statement

Constrained Optimization Problem: $\min\{f(s) | s \in \mathcal{F}\}$

- $\mathcal{F} \subseteq \mathcal{S}$ set of **feasible solutions**
- $\mathcal{S}$ set for **candidate solutions** (combinatorial structures)

The concept of feasibility is flexible and a design choice. Most typically, it implies satisfying the constraints of the problem.

Guiding rules:

- if it has an objective function value, then it is a feasible solution

- constructive search algorithms work with partial, infeasible solutions

2.1 Constraint Handling

Constraints in heuristic methods are handled:

- **Implicitly** in the definition of the **combinatorial structures** that constitute the search states: assignments, permutations, (sub)sets, partitions, (sub)graphs, sequences, set of sequences, ...
 - As **one-way** constraints between variables
 - As **soft constraints** ie, relaxed in the evaluation function as penalty components with large weights or as lexicographically more important components

2.2 A Classification

White box optimization:

- models can be expressed mathematically

Gray box optimization:

- internal information about objective function computation is often available models that have a mathematical expression but may need data to determine them (eg, neural networks)

Black box optimization:

- no mathematical expression is available
- We assume a zeroth order oracle that takes as input a point $s \in S$ and outputs a dual bound if infeasible or the value of f at s .

2.3 Approaches to ROHs

White/Gray box:

- representation (modelling) + reasoning (search) constraint based local search, comet, local solver (Hexaly)

Black box:

- a different approach, framework separating problem from solvers and defining the interface specification

⇒ COST Action: ROAR-NET



Figure 6: <https://roar-net.eu>

Definition (Search or Optimization Algorithm)

Goal formulation: we want to find the minimum with respect to some criterion from a set of candidate elements.

Problem formulation: Given a description of the states, an initial state and actions necessary to reach the goal, find a sequence of actions to reach the goal.

Search: the algorithm simulates sequences of actions in the model of the goal, searching until it finds a sequence of actions that reaches the goal. The algorithm might have to simulate multiple tentative answers that do not meet the goal, but eventually it reaches a solution, or it will find that no solution is possible.

2.4 Components of a Search Algorithm

(valid for complete or heuristic and constructive or perturbative algorithms)

- **State** (or *candidate*) solution: A definition of states of the search.
- **Search Space:** The set of possible states
- **Initial State:** Can be specific state ($x_0 = [0, 0]$) or a random initialized state
- **Goal:** A set of one or more goal states. Sometimes there is one goal state sometimes there is a small set of alternative goal states
- **Evaluation function:** $f(s)$ assesses the distance from a potential goal. (*note: different from “objective”, it can also include penalties due to constraint violations*).
- **Action type t :** available to the algorithm.
- A finite set of **actions** of type t that can be executed in s , $\text{Actions}(t, s)$.
- A **transition model** that describe what each action t does. $\text{Result}(s, a)$ returns the state that results from doing action $a \in \text{Actions}(t, s)$ in state s .

- An action-cost function, $\text{Action-Cost}(s, a, s')$, that gives the numeric cost of applying action a in state s to reach state s' .

3 Constructive Search

Recall: Complete Graph Search Methods

- uninformed settings
 - Breadth-first search
 - Uniform-cost search
 - Depth-first search
- informed settings
 - Greedy best-first search
 - A^* search

3.1 Complete Tree Search

Search Space

Tree with branching factor at the top level nd at the next level the branching factor is $(n - 1)d$.
The tree has $n! \cdot dn$ leaves even if only dn possible complete assignments.

3.2 Exploiting Information

Assessment of partial, infeasible solutions

The priority assigned to a node x is determined by the function

$$f(x) = g(x) + h(x)$$

$g(x)$: cost of the path so far $h(x)$: heuristic estimate of the minimal cost to reach the goal from x

A **greedy best-first search** uses h to decide while a A^* best-first search, is cost optimal when $h(x)$ is an

- admissible heuristic: never overestimates the cost to reach the goal
- consistent: $h(n) \leq x(n, a, n') + h(n')$ (*consistent* $\Rightarrow$ *admissible*, only necessary in graph search)

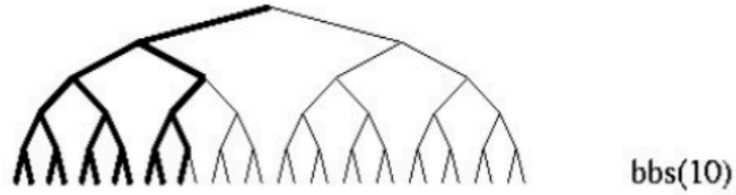
3.3 Greedy algorithms

3.4 Incomplete Search algorithms

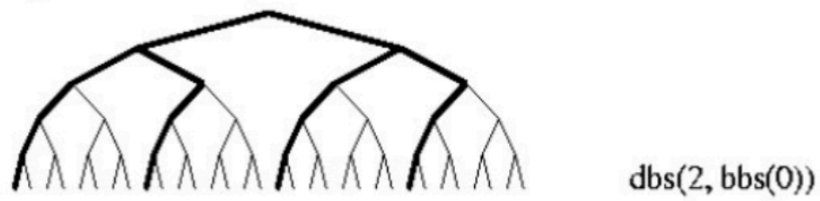
These can either be implemented on the 'backtracking' framework or outside. If they are implemented on the backtracking framework, we can move around truncating paths, and given enough time, this will complete. This is not guaranteed by those implemented outside.

3.5 Bounded backtrack

Bounded-backtrack search:



Depth-bounded, then bounded-backtrack search:



Visualizations by Helmut Simonis

Figure 7:

3.6 Incomplete Search Ideas

3.7 Construction-Based Metaheuristics